RPX

RPA/RPX Kernel

精确重构证明规范手册

Reformulation Proof Assembly
Reformulation Proof eXecutable

完整模型 · 优化等价 · 推导与联合扩展
文本汇编 · 二进制格式 · 完整证书
Loader · 执行语义 · 异常与符合性

当前实现参考 · MCM 1 · RPX 11 · RPA 12

2026 年 10 月 17 日

依据当前安全 Rust kernel 的实现重写。
证书仅含正向、反向两个完整证明块。

阅读约定与文档状态

RPX 的唯一用途是证明两个完整 MILP 的优化等价。调用者提供源模型、目标模型与一份证书。证书恰含 forward、reverse 两个 Block，不含可选择的正确性要求。RPA 12 是行式证明汇编，直接降低为 RPX 11 字节；每条逻辑指令恰好对应一个 Step、End 或 DerivationEnd。旧 RPA 11 JSON 与 RPA-S 1 均被拒绝，不再有命名检查、模型绑定或模型路径选项。MCM 1 保存完整精确模型。

本手册于 2026 年 10 月 17 日同步 RPA 12 汇编。当前内核只接受 RPX 11 与 RPA 12，不接受旧版本文本头。历史 V1–V10 文件及验收记录保留原义；迁移工具使用独立旧内核提取双向完整 MI 构造，然后必须由当前内核重新验证。升级版本号本身不构成迁移。

规范与实现

“必须”“拒绝”描述当前格式、数学接受条件和公开行为；测试与测量是证据而非形式化证明。完整字节规范为 `docs/FORMAT.md`，数学语义为 `docs/SEMANTICS.md`。Rust 数据结构在 `proof.rs`，执行器在 `verify.rs`，编解码在 `codec.rs`，五类构造在 flow、interval、menu、signed、subflow 模块。

可信边界包括精确模型导入、算术、作用域、义务生成、推理规则和联合存在引理的实现。作者程序、编译器、求解器结果、文件名和哈希不能授予数学事实。成功汇编只说明可编码；只有对实际完整模型的验证成功才表示优化等价。

阅读顺序

先读第一章的唯一目标和第二章的执行语义，再查第三章的推理规则。第四章与第五章说明格式、资源和文件边界；第六章及第七章给出编写与运行方式。第八章记录简化后的测量，附录A说明迁移。

内核不进行证明搜索或 MILP 求解，也没有 solver 数值解恢复命令。验证失败或资源不足不代表重构在数学上不成立；证明语言可能无法紧凑表达所需构造。

目录

阅读约定与文档状态 **i**

第一章 完整模型与优化等价 **1**

1.1 完整模型与精确数域 1

1.2 规范仿射表达式与事实 2

1.2.1 三种事实 2

1.2.2 模型事实 2

1.3 唯一验收目标 2

1.4 共同扩展的逻辑边界 3

第二章 执行语义与验收 **4**

2.1 引用解析与局部定义 4

2.1.1 父子作用域 4

2.2 目标命题检查与 Exact 4

2.3 步骤序列与穷尽分类 5

2.3.1 Cases 的共同事实 5

2.3.2 Split 的完整子证明 5

2.4 完整 Return 与义务生成 5

2.5 lift 的执行协议 6

2.6 完成、资源和拒绝 6

2.7 正确性不变式 6

第三章 操作与推理规则参考 **7**

3.1 操作总表与共同条件 7

3.2 Lin：精确线性组合 7

3.3 Round：整数表达式精确舍入 8

3.4 Drop：局部可见性删除 8

3.5 EqPair：精确反对称性 8

3.6 Cases 与 DerivationEnd：局部穷尽证明 8

3.7 Plotkin：带已验近对 clique 的码上界 9

3.8 Return、Split 与 Absurd 9

3.8.1 Return：完整输出 9

3.8.2 Split：穷尽完整子 Block 9

3.8.3 Absurd：恒定矛盾 10

3.9 共同扩展的公共约定 . . . 10
3.10 FlowLift：共同源商品分解 . . . 10
3.11 IntervalLift：独立实区间选择 . . . 11
3.12 MenuLift：可选二元菜单 . . . 11
3.13 SignedLift：有激活的有界实数正负分解 . . . 11
3.14 SubflowLift：有需求部分的共同有界子流 . . . 12

第四章 MCM 与 RPX 二进制格式 13
4.1 公共记录 . . . 13
4.2 唯一证书布局 . . . 13
4.3 操作载荷 . . . 13
4.4 结构限制 . . . 14

第五章 文件边界、API 与资源 15
5.1 完整文件验收 . . . 15
5.2 编写接口 . . . 15
5.3 资源与诊断 . . . 15

第六章 文本汇编与编写 16
6.1 文件、词法与逻辑指令 . . . 16
6.2 引用、坐标与精确表达式 . . . 16
6.3 Step 指令 . . . 17
6.4 分支头与终结指令 . . . 17
6.5 编写示例 . . . 19
6.6 规范反汇编与命令行 . . . 20

第七章 完整可复现实例 21
7.1 等式消元 . . . 21
7.2 冗余行与分段恢复 . . . 22
7.3 Python 编译器 API . . . 22
7.4 统一检查命令 . . . 23

第八章 性能与证据范围 24
8.1 RPX 11 简化对比 . . . 24
8.2 回归证据 . . . 24

附录 A 版本边界与迁移 25

第一章　完整模型与优化等价

本章规定当前 kernel 的数学对象与成功验证所证明的命题。RPA（Reformulation Proof Assembly）12 是行式证明汇编文本，RPX（Reformulation Proof eXecutable）11 是相应的二进制证书；RPA 12 直接降低为 RPX 11，两者描述相同的证明对象。模型格式为 MCM 1。

1.1　完整模型与精确数域

一个模型为

$$M = (V, R, f), \quad V = (v_0, \ldots, v_{n-1}), \quad R = (r_0, \ldots, r_{r-1}),$$

其中每个变量给出整数声明及可选上下界，每个行给出一个仿射表达式及可选上下界，f 为完整的规范化最小化目标。变量与行分别从零编号；编号指向实际向量，名称只作元数据。验证器接收调用者提供的两个**完整**模型，必须检查全部结构和数值。RPX 没有用模型摘要、选定行集合或模型名称代替这些模型的机制。

全部有限标量属于精确数域 $\mathbb{Q}$，采用

$$q = p/d, \qquad p \in \mathbb{Z}, \quad d \in \mathbb{Z}_{>0}, \quad \gcd(|p|, d) = 1.$$

零的规范值为 0/1。分母为零、非规范分数及超过资源界的标量拒绝。十进制、科学计数法和分数字面量均表示精确有理数；例如 `0.125` $= 1/8$，`1e-6` $= 1/1000000$。模型系数为有理数不意味着连续坐标也限于有理数：证明必须对实数域成立。**必须：**决定接受的加法、乘法、取整、代换及比较遵循精确算术，不使用可行性容差。

变量 v_i 表示 $x_i \in \mathbb{R}$，带条件

$$l_i \le x_i \le u_i,$$

其中缺失下界或上界分别表示负无穷或正无穷。仅实际存在的界参与事实与义务。变量 domain 只有连续与整数两种；所谓二元变量是声明整数且具有相应有效界的变量，没有第三种 MCM domain 标签。行 j 表示

$$L_j \le a_j(x) \le U_j,$$

a_j 可以带常数，可以为空支持；缺失的一侧不产生条件。两侧有限且相等时是等式行。行常数不必移入界。**必须：**矛盾的变量界或行界仍是合法模型数据，表示可能为空的可行域；验证器不得修复或删除它们。

目标为

$$f_M(x) = c_M + \sum_i d_{Mi} x_i.$$

导入最大化模型时，整个目标连同常数取负。目标常数进入完整目标比较。MPS 与 LP 导入属于可信转换边界；不支持的原始文件构造应在导入时拒绝，不能在证明中赋予其未定义的含义。

模型名称、变量名称和行名称必须非空、为 ASCII、无控制字节且不超过 1024 字节。变量名称在变量向量内唯一，行名称在行向量内唯一；两个命名空间独立。名称的唯一性是结构条件，名称本身不授予任何数学事实。

1.2 规范仿射表达式与事实

表达式为

$$e(x) = c + \sum_{k=0}^{t-1} a_k x_{i_k}, \quad 0 \le i_0 < \cdots < i_{t-1} < n, \quad a_k \neq 0.$$

常数与系数为规范有理数。空支持表示常数表达式。序列必须已经严格有序、无重复、无零系数；二进制解码与证明验证不会隐式修复输入表达式。内部算术可合并同坐标并消去零项，所得结果仍须规范且满足资源界。表达式直接作为记录出现，没有 ExprId 池、隐式变量名或映射寄存器。局部证明表达式可使用当前扩展模型坐标。

两个表达式相等是常数与全部坐标系数的精确相等。逻辑上等价的 $e \le 0$ 与 $2e \le 0$ 不自动成为同一 proposition。哈希只可帮助查找，不能代替精确表达式比较。

1.2.1 三种事实

类型	数学意义
`Le`	$e(x) \le 0$，记为 $\mathrm{LE}(e)$。
`Eq`	$e(x) = 0$，记为 $\mathrm{EQ}(e)$。
`Int`	$e(x) \in \mathbb{Z}$，记为 $\mathrm{INT}(e)$。

事实在产生时具有作用域：当前模型、当前路径假设以及活跃局部定义共同决定其有效域。一个经过检查的 $\mathrm{EQ}(e)$ 可以在目标命题检查处证明同表达式的 $\mathrm{LE}(e)$ 或 $\mathrm{INT}(e)$，因为零是不正整数。这不改变各条指令的原始前提种类要求；例如 Round 仍须一个 Le 和一个 Int。证明语言不含一般严格不等式事实。

1.2.2 模型事实

当前输入模型的静态引用按下式生成：

$$\begin{aligned}
&\texttt{VarLower}(i): && \mathrm{LE}(l_i - x_i),\\
&\texttt{VarUpper}(i): && \mathrm{LE}(x_i - u_i),\\
&\texttt{VarInt}(i): && \mathrm{INT}(x_i),\\
&\texttt{RowLower}(j): && \mathrm{LE}(L_j - a_j(x)),\\
&\texttt{RowUpper}(j): && \mathrm{LE}(a_j(x) - U_j),\\
&\texttt{RowEq}(j): && \mathrm{EQ}(a_j(x) - L_j).
\end{aligned}$$

界引用要求相应有限界存在；RowEq 要求两侧有限且相等。VarInt 要求变量声明整数。引用无效编号、缺失界或不满足这些条件时拒绝。当前模型包括 lift 子体中的扩展行；新行的权威来自相应可信规则，不能在规则自己的前提证明中使用。

1.3 唯一验收目标

令 P_M^{MI} 表示完整模型 M 的混合整数可行域。证书恰有两棵证明树：

$$\text{forward}: \ \forall x \in P_S^{MI} \ \exists y \in P_T^{MI} : f_T(y) \le f_S(x),$$

$$\text{reverse}: \ \forall y \in P_T^{MI} \ \exists x \in P_S^{MI} : f_S(x) \le f_T(y).$$

两个方向都没有根域限制，始终使用全部实际整数声明。每个叶子必须由同一个完整输出向量满足全部输出条件。Return 给出仿射向量；Split 允许分支采用不同向量；lift 先共同选择辅助坐标。

两个总构造首先给出同时可行或不可行。对非空域，正向给出 $\inf f_T \le \inf f_S$，反向给出反向不等式，故扩展最优值相同。约定空域下确界为 $+\infty$，目标向下无界时为 $-\infty$。如果 y^* 是目标模型的最优解，则恢复的可行 x 满足 $f_S(x) \le f_T(y^*) = \inf f_S$，因而也是最优解。

此证明方式是优化等价的充分条件，不是证书语言完备性的声明。它不要求双射、LP 等价、每个可行点的目标值相等或指定坐标保持。例如 $\min\{x : x \in \{0,1\}\}$ 可变为单点 $x = 0$，正向取常数零，反向取恒等。不存在可配置 Contract、LP 模式、projection、inverse、点性质或 improvement 字段。事实 Eq 仍可证明相同表达式的 Le，因此目标恰好相等的构造也满足唯一目标。

数学构造可恢复最优解，并不表示当前工具已经提供 solver 可调用的数值恢复程序。目前 kernel 检查构造的合法性，不执行输入解上的 lift 选择与恢复。

1.4 共同扩展的逻辑边界

各 lift 的核心是可信存在命题

$$M(x) \wedge \text{路径假设} \wedge \text{已验前提} \implies \exists z\ E(x,z).$$

它保留旧模型与目标，仅向当前子体加入新坐标、界和行。每项前提都在同一个扩展前作用域证明，所有项目的选择同时存在。子体必须再对一个共同输出向量证明整个实际输出模型与目标不增义务；不能用每条行各自不同的存在选择拼出可行性。具体数学前提、生成顺序与构造论证见第三章。

这些规则是可信推理代码，不是动态定理注册、求解器回调或外部插件。成功检查前提、验证生成模型结构及运行子体，均不独立证明扩展引理或 Rust builder 正确。数学论证、独立构造 oracle 和攻击测试是不同证据，不能宣称它们构成实现的形式化证明。

第二章　执行语义与验收

Certificate 恰含 forward 与 reverse。每个 Block 先执行有序 Steps，再执行一个 End。两个根均从各自完整模型、空局部事实和空路径假设开始，共享同一个工作预算。第一方向成功不会给第二方向增加事实，任一方向失败均不接受整个证书。Block 证明完整输出构造；Derivation 只为 Cases 证明一条共同事实。

2.1　引用解析与局部定义

九种 Ref 的完整语义如下。编号均是相应逻辑空间的索引，而非文件偏移。

引用	解析结果与有效条件
`VarLower`(i)	当前 I 的 LE($l_i - x_i$)；须存在有限下界。
`VarUpper`(i)	当前 I 的 LE($x_i - u_i$)；须存在有限上界。
`VarInt`(i)	当前 I 的 INT(x_i)；须声明整数。
`RowLower`(j)	LE($L_j - a_j$)；须有有限下侧。
`RowUpper`(j)	LE($a_j - U_j$)；须有有限上侧。
`RowEq`(j)	EQ($a_j - L_j$)；须有相等的两有限侧。
`Local`(id)	Δ 中当前可见的已验定义；已 Drop、未定义或仅在兄弟可见时拒绝。
`Assumption`(k)	Γ_k；0 为最早加入的假设，越界拒绝。
`Exact`	只对 checker 已确定的目标 proposition 做窄规则检查，不解析为普通事实。

Lin、Round、EqPair、Cases、Plotkin 用显式 u32 `id` 发布一个局部事实。ID 不须从零连续编号，也不须递增。发布时该 ID 不得仍然活跃，且活跃数不得超过预算；所有前提、类型和表达式先通过检查。Drop 按其数组顺序删除当前可见 ID，缺失 ID 或同一数组中重复删除已删 ID 均拒绝。一个 ID 仅在删除后才能复用，新的定义有新的 proposition；引用只读当前定义。

2.1.1　父子作用域

Split 和 Cases 的两个子作用域继承父局部事实与既有假设。子体可删除继承 ID，在该子体内复用 ID，并创建其他事实；这些变化不能改变父或兄弟。两子体各附加本分支假设，退出时撤去该假设。Cases 两子体都成功后，仅在父作用域发布其指定的一条共同结论；子体的其他事实、删除或重定义均不向父发布。Split 是端点，不存在需要合并的父续体。

lift 把当前活跃事实和路径假设带入其唯一子体。旧坐标和旧行的完整前缀不变，因而旧事实仍有效；所有新行只在子体可见。在分支内扩展不会给兄弟增加坐标或行。实现可共享不可变父状态并保存子差量；这种存储选择不能改变上述可见性。

2.2　目标命题检查与 Exact

设 checker 要证明 (K, e)，通常引用 r 解析为 (K_r, e_r)。接受当且仅当 $e_r = e$ 精确成立，且 $K_r = K$ 或 $K_r =$ `Eq`。比例相等、仅在某个点取值相等、ID 相等及显示文本相等均不足。

Exact 不从证书读入命题；checker 先构造 e，再检查：

1. EQ：e 无变量项且常数为零。
2. LE：e 无变量项且常数不正。
3. INT：常数为整数，每项系数为整数，且每个出现坐标声明整数。

常数整数没有坐标前提。非恒真的仿射不等式不能通过 Exact；无模型整数性支持的非恒定表达式也不能通过 Exact。生成的菜单或 signed 二元坐标可满足 INT 的窄规则，生成的 flow、interval 和 subflow 连续坐标不能。

Exact 可出现在 Return 检查、lift 前提和 DerivationEnd Witness。它不能作为 Lin（即使乘子为零）、Round、EqPair、Plotkin、整数 Split/Cases 或任何 Absurd 的原始事实操作数。一个已有 Eq 可做目标检查的种类弱化，但不赋予原始操作数自动种类转换。

2.3 步骤序列与穷尽分类

步骤顺序执行；后一步仅能引用当前有效的模型事实、假设与前面活跃定义。各条推理规则见第三章。六种 Step 的命名空间与八种 End、两种 DerivationEnd 的命名空间独立。同一个数字 tag 在不同位置不表示同一操作。

Split 与 Cases 都先验证规范表达式 e。未提交整数引用时，两个分支假设为

$$\mathrm{LE}(e), \qquad \mathrm{LE}(-e).$$

它们覆盖全实数域，$e = 0$ 的边界同时属于两支。提交整数引用时，须解析出种类确为 *Int* 且表达式恰为 e 的事实，然后两假设为

$$\mathrm{LE}(e), \qquad \mathrm{LE}(1 - e).$$

整数域中分别表示 $e \le 0$ 与 $e \ge 1$，穷尽且互斥。不能用 Exact 或 Eq 原始引用冒充该 Int 前提。

2.3.1 Cases 的共同事实

Cases 在父作用域指定 ID、Kind 和 expression；两 Derivation 分别在穷尽分支上证明这同一 proposition，或用普通 Absurd 证明该支为空。Witness 使用目标命题检查规则，因此允许 Exact 以及同表达式的 Eq 种类弱化。两支不能各自选择不同 expression，再要求 checker 猜测共同结论。Derivation 没有 map、Return、lift。内部 Steps 仍可递归 Cases；成功后父得到唯一指定事实。

2.3.2 Split 的完整子证明

每个 Split 子 Block 都必须以该上下文允许的端点成功完成。两支 Return 可以给出不同完整输出向量；Absurd 可关闭已证明不可能的支。不能把一个分支只验了变量界、另一分支只验了行的证明合成一个 Return。判为不可能的支仍须先证明合法矛盾；kernel 不按猜测的可达性跳过证明。

2.4 完整 Return 与义务生成

map 长度恰等于输出变量数，表达式使用当前输入坐标并须规范。对同一个完整向量 Y，checks 依次证明：

1. 输出变量按坐标序：先有限下界 $l_j - Y_j \le 0$，再有限上界 $Y_j - u_j \le 0$，然后对每个实际整数变量证明 $Y_j \in \mathbb{Z}$。

2. 输出行按行序：两有限侧相等时证明 $a_j[Y] - L_j = 0$；否则先证明有限下侧，再证明有限上侧。不存在的界不产生义务，矛盾界仍须检查。
3. 最后一项永远为 $f_O[Y] - f_I \le 0$，包含两个完整目标的常数。

每项引用必须证明准确表达式与种类，缺失、多余或错误顺序均拒绝。等式事实可以证明同表达式的不增目标；没有 Equal/None 模式。每个叶子的一套义务始终使用一个共同 Y。

2.5 lift 的执行协议

五种 lift 均先检查 witness、尺寸与资源，再在扩展前作用域核验全部前提，最后进入生成模型上的唯一 body。旧模型的变量、行和目标保持完整前缀，新变量目标系数为零。新事实只在 body 可用，不能循环证明本次 lift 的前提。嵌套 lift 可用上层已经建立的坐标。存在引理保证一个共同扩展点满足全部新条件；内核构造公式，不运行数值选择或路径搜索。

2.6 完成、资源和拒绝

完整解码先于执行，所有分支字节均须合法。普通 Absurd 只接受正的恒定 Le 或非零恒定 Eq。每个 Block 可以 Return、Split、Absurd 或五种 lift 完成；Cases 的 Derivation 只能 Witness 或 Absurd。没有附加性质阶段，没有可跳过的方向，也不以猜测不可达为由略过失败分支。

默认工作预算为 10^9、活跃事实预算为 10^6、组合嵌套深度为 64。两个方向共享工作预算，局部事实分别重建。Split、Cases 和 lift body 增加深度。超出工作、深度、位数、项数或分配限制均不接受；资源不足不是数学反例。诊断只报告 direction、step、obligation 等已知位置，不参与决定接受。

2.7 正确性不变式

对完整输入域与路径假设归纳：可见事实有效；每个分类穷尽；矛盾叶没有可行点；每个 lift 对每个父点给出共同可行扩展；每个 Return 证明完整输出可行且目标不增。因此每个成功方向给出一个总构造，两个方向共同推出优化等价。这依赖精确算术、模型导入、作用域、义务生成与可信构造规则的实现正确性，不声称 Rust 程序已由证明助手形式化验证。

第三章　操作与推理规则参考

本章逐条定义当前六种 Step、两种 DerivationEnd 与八种 End 的前置条件、精确语义、状态变化及拒绝条件。Kind 为 Le/Eq/Int，Ref 的九种形式与 Exact 规则见第二章。载荷是逻辑字段，不是本章另定义的 opcode 布局；逐字节记录、保留位见第四章。

3.1　操作总表与共同条件

Step tag	名称	作用
0	Lin	产生一种精确线性组合事实。
1	Round	结合 LE 和 INT 得到精确舍入不等式。
2	Drop	删除显式 ID 列表。
3	EqPair	由相反 LE 得到 EQ。
4	Cases	两穷尽局部证明产生一条共同事实。
5	Plotkin	静态二元码距离上界。

End tag	名称	作用
0	Return	完整共同输出向量和全部义务。
1	Split	两穷尽完整子 Block。
2	Absurd	恒定矛盾关闭当前域。
3	FlowLift	共同源商品流分解。
4	IntervalLift	同时选择实区间坐标。
5	MenuLift	可选整数菜单的二元展开。
6	SignedLift	有激活的有界实数正负分解。
7	SubflowLift	同时保留有需求的有界子流。

产生事实的 Step 在成功后将 (kind, e) 发布至显式 `id`。所有表达式必须在当前 input 维度规范，标量及运算中间值必须精确且满足资源界。活跃 ID 不得覆盖；拒绝任何前提引用失活、错误种类或不匹配表达式。Drop 不产生事实，End 不向父作用域发布局部事实。证书没有 NOP、逐项 MAP、单独 CHECK、RET、YIELD 或动态定理调用操作。

3.2　Lin：精确线性组合

字段为 `id`、`kind`、精确 `constant` c 与有序 `terms` (r_k, λ_k)。先解析每个 r_k 为事实 (K_k, e_k)，构造并规范化

$$e = c + \sum_k \lambda_k e_k.$$

允许条件依输出 Kind 分别为：

输出	前置条件
Le	$c \le 0$；每个前提为 Eq（乘子任意有理数），或 Le 且 $\lambda_k \ge 0$。
Eq	$c = 0$；每个前提均为 Eq，乘子任意有理数。
Int	$c \in \mathbb{Z}$；每个前提为 Eq（乘子任意有理数），或 Int 且 $\lambda_k \in \mathbb{Z}$。

语义是对有效不等式做非负组合、对零等式做任意组合，或对整数值做整数组合。INT 中 rational 倍 Eq 仍等于零，不要求该倍数为整数。空 terms 可产生常数不大于零的 LE、零 EQ 或常数 INT。常数不满足对应条件、LE 的负不等式乘子、EQ 的非等式前提、INT 的分数整数乘子均拒绝。**必须：**即使 $\lambda_k = 0$，也须解析并检查该前提的引用和种类；零乘子不隐藏无效引用或 Exact。成功后只发布这一条规范 e；本指令不会自动把正比例相等的事实视为相同。

3.3 Round：整数表达式精确舍入

字段为 `id`、`le` 与 `integer`。两原始引用须分别为

$$\mathrm{LE}(a), \qquad \mathrm{INT}(z), \qquad a = c_a + \sum_i a_i x_i, \quad z = c_z + \sum_i a_i x_i.$$

变量项列表必须精确相同，常数可不同。令 $d = c_a - c_z$，发布

$$\mathrm{LE}\big(z + \lceil d \rceil\big).$$

因为 $a \le 0$ 给出整数 $z \le -d$，故 $z \le \lfloor -d \rfloor = -\lceil d \rceil$。取整作用于精确有理数，对负数也使用数学上取整。前提不是 Le/Int、变量支持或系数不同、引用无效、Exact 以及任何超限中间数均拒绝。上界和下界舍入共用这一个规则；下界须先把相应表达式改为负号形式，没有额外 ROUND.LO opcode，也没有未证整数表达式的自动舍入。

3.4 Drop：局部可见性删除

字段为 `ids`，是一个显式 u32 数组，不是半开寄存器区间。按数组顺序删除各个当前可见 Local ID；空数组有效。删除继承事实时只在当前子作用域隐藏它，父和兄弟仍可见。成功删除后该 ID 可被后续定义复用。引用缺失或已删除的 ID 时拒绝，同一 Drop 中重复 ID 也会因第二次删除不存在事实而拒绝。该指令不删除模型事实或假设，也不修改模型、map 或任何已验数学命题。

3.5 EqPair：精确反对称性

字段为 `id`、`positive` 和 `negative`。两引用须种类均为 Le，表达式分别精确为 e 与 $-e$。发布 $\mathrm{EQ}(e)$，保留 positive 的表达式方向。前提为 Eq、仅按正比例相反、常数不相反、Exact 或引用失活时均拒绝。语义为 $e \le 0$ 且 $-e \le 0$ 推出 $e = 0$；若后续需 $\mathrm{EQ}(-e)$，须用 Lin 显式乘以 -1，不由该操作隐式改向。

3.6 Cases 与 DerivationEnd：局部穷尽证明

字段为 `id`、`kind`、`expression` p、`split` e、可选 `integer`、以及 `left`、`right` 两 Derivation。p, e 在父维度验证。无 integer 时加入 $\mathrm{LE}(e)$ 与 $\mathrm{LE}(-e)$；有 integer 时须原始 Int 引用恰证明 e，加入 $\mathrm{LE}(e)$ 与 $\mathrm{LE}(1 - e)$。

每个子证明先执行自己的 Steps，随后使用独立 DerivationEnd 命名空间：

tag	端点	条件
0	Witness(Ref)	按目标检查规则证明恰为 $(\texttt{kind}, p)$，允许相同 Eq 及合法 Exact。
1	Absurd(Ref)	证明无变量项且常数为正的 Le，或非零常数 Eq；不得使用 Exact。

两支隔离，均成功才在父发布 $(\texttt{kind}, p)$；子内部定义不会泄漏。若两支没有同一指定结论、不合法关闭矛盾、覆盖父活跃 ID、integer 不是准确 Int、任一子证明失败或深度超限，整个 Cases 拒绝。它可以嵌套 Cases，却不能加入输出 map、lift 或严格改进端点。其结论的有效性来自穷尽覆盖，不是把父事实库与两个子事实库合并。

3.7 Plotkin：带已验近对 clique 的码上界

字段为 `id`、`bits` b、`distance` d、`variables` (i, ℓ_i) 和 `cliques`。要求 $1 \le b, d \le 64$ 且 $2d > b$。选中坐标不得重复，实际变量须声明整数、具有有限下界 ≥ 0 与有限上界 ≤ 1。每个 u64 label 只能使用低 b 位；$b = 64$ 时全部位可用。label 不要求互异，但相同 label 的选中变量必须由近对前提约束。

每个 clique 原始引用必须是严格如下形式的 Le：

$$\mathrm{LE}\left(\sum_{i \in C} x_i - 1\right),$$

所有出现系数均为 $+1$，常数为 -1，每个出现坐标的实际有限下界不负。clique 可含未选中的其他非负坐标，它们不要求二元。对选中变量的每一对 (i, j)，若 $\mathrm{popcount}(\ell_i \operatorname{xor} \ell_j) < d$，至少一个已验 clique 必须同时包含两者。clique 错种类、错误常数/系数、负下界、Exact 或任何未覆盖近对均拒绝。

发布

$$\mathrm{LE}\left(\sum_{i \text{ selected}} x_i - \left\lfloor \frac{2d}{2d - b} \right\rfloor\right).$$

证明：在任意整数可行点，激活的 k 个二元变量的 labels 两两距离至少 d。总两两距离至少 $dk(k-1)/2$；按位计，每位贡献至多 $k^2/4$，所以总距离至多 $bk^2/4$，推出所示界（$k = 0, 1$ 也成立）。这是有限静态可信推理规则，不依据图名称或外部求解结果。未选择变量或没有 clique 也按相同检查执行，近对不存在时覆盖要求为空真。配对数量、clique 成员位表和扫描工作另受资源界约束。

3.8 Return、Split 与 Absurd

3.8.1 Return：完整输出

字段为 `map` 和 `checks`。map 长度恰为实际输出维度，所有表达式使用当前输入坐标并规范。checks 恰按第2.4节完整义务顺序逐项证明。成功即完成该叶的完整存在构造，不产生继续执行的位置。缺失/多余 map、错误维度、缺失/多余 checks、任一义务不匹配均拒绝。输出条件不得用不同的 map 元组分别证明。

3.8.2 Split：穷尽完整子 Block

字段为 `expression` e、可选 `integer`、`left`、`right`。表达式与整数引用条件和 Cases 完全相同。两子 Block 继承父事实、各加入一条分支假设，并依序全部执行。每个子体须成功完成当前上下文允许的完整端点。成功表示每个父点至少属于一个已完成子域；不合并 map 或部分检查项。引用/表达式错误、非 Int 整数前提、失败子体或深度超限均拒绝。没有专门的二元枚举、有限区间或多路 split opcode；需要时可用规范 e 与二叉树表达。

3.8.3 Absurd：恒定矛盾

字段为 `fact`。解析为当前事实后，要求表达式变量项为空，且为 $\mathrm{LE}(c)$、$c > 0$ 或 $\mathrm{EQ}(c)$、$c \neq 0$。成功关闭当前域的空支，不生成输出向量。Int、非恒定式、非矛盾常数、失活引用和 Exact 均拒绝。DerivationEnd Absurd 使用同一矛盾谓词。

3.9 共同扩展的公共约定

五种 lift 均为 End，载荷含 witness 和一个 body Block。记进入端点时变量数为 N、行数为 R。每个 witness 表达式只能使用这些旧坐标；全部组的全部前提先在旧模型与旧事实作用域检查，才允许 body 访问生成模型。原模型名称、变量/行向量与目标保持完整前缀。新名称避碰，不决定编号。对生成行的引用遵守普通 RowLower/RowUpper/RowEq 条件。

各 builder 检查 count 加乘、端点编号、规范表达式、有理数中间值、扩展维度与项数，并在剩余共享预算内执行 fallible allocation 与名称生成。witness 预备成功只得到候选模型与待证前提；不表示前提已经有效。任何前提错误、作用域错误、深度或资源超限均拒绝。一旦进入 body，全部新变量和全部新行可同时使用，原事实和路径假设继续有效。

以下存在论证覆盖实数输入点，不只覆盖可枚举的有理样本。生产 checker 构造公式，不执行这些数学论证中的搜索或数值选择。

3.10 FlowLift：共同源商品分解

字段为 `witness`、`body`。FlowWitness 给出 `nodes`、有序 `arcs` $(\mathrm{tail}, \mathrm{head})$ 和非空 `groups`。节点数 $n \geq 1$，所有图端点在 $[0, n)$；允许平行弧、环和零弧。每组给出 `source` s、非空 `commodities` (t_k, d_k)、每弧的 `flows` F_e、`nonnegative` 引用和 `balances` 引用。每个 $d_k \geq 0$；允许重复终点、$t_k = s$ 和零需求。flows/nonnegative 长度必须恰为弧数 m，balances 恰为节点数 n。

定义散度为出流减入流，环在散度中抵消；令

$$b_v = \sum_k d_k(\mathbf{1}_{v=s} - \mathbf{1}_{v=t_k}).$$

每组按序要求先所有弧 $\mathrm{LE}(-F_e)$，然后所有节点

$$\mathrm{EQ}\left(\sum_{e:\,\mathrm{tail}=v} F_e - \sum_{e:\,\mathrm{head}=v} F_e - b_v\right).$$

零式和冗余节点仍须提供有效引用。

每组按 commodity、arc 顺序追加连续 $z_{k,e} \geq 0$，无有限上界。若本组起始坐标为 S，则 $z_{k,e}$ 的编号为 $S + km + e$。先按 commodity、node 顺序追加

$$\mathrm{div}(z_k)_v = d_k(\mathbf{1}_{v=s} - \mathbf{1}_{v=t_k}),$$

再按 arc 顺序追加 $\sum_k z_{k,e} - F_e = 0$。每组 K 个 commodity 增加 Km 变量、$Kn + m$ 行。商品平衡行常数零，右侧为两个相等界；弧总量行保留 $-F_e$ 的完整常数、两界零。空弧图仍保留全部商品节点等式。

共同存在。 固定满足前提的父点，从残余非负 F 逐商品提取正残余的简单源到终点路径。未满足的终点必可到达：否则能到该终点的正残余可达集合不含源、没有正残余入弧，总散度应非负，而剩余终点需求给出负总散度，矛盾。每次减去路径瓶颈与剩余需求的较小值；一次完成一个商品或永久消去一弧，有

限步终止。零需求或源等于终点无需路径。最后残余是环流，将全部残余加入第一个商品，保留所有弧总量，含自环及断开的环流。各组新坐标互不相交，故选择共同存在。

这不授予每商品上界、整数性或异质成本关系；生成 z 全为连续。缺失商品、负需求、图端点错误、错误前提数或旧维度之外 F 均拒绝。输出中的额外容量、目标与投影必须在 body 另证。

3.11 IntervalLift：独立实区间选择

字段为 `intervals`、`body`。每项 IntervalWitness 为有序 `lower` L_i、`upper` U_j 与 `checks`。checks 长度须等于两列表长度的精确乘积，依 lower-major 次序证明

$$\mathrm{LE}(L_i - U_j) \quad \text{对每个} (i, j).$$

无 lower 或无 upper 时不产生配对前提。所有项可共享父坐标，但不能引用同一次 lift 新增的坐标。

按项序追加一个无固有有限界的连续坐标 z，第 k 项编号 $N + k$。行按项序，先 lower 列表的 $z - L_i$ 下界零行，再 upper 列表的 $z - U_j$ 上界零行；另一侧缺失，表达式保留负旧仿射常数。自由区间仍追加变量；空 intervals 是恒等扩展。

共同存在。 有 lower 时取其有限列表最大值，配对前提保证它不超过所有 upper；无 lower 有 upper 时取 upper 最小值；两列表皆空取零。每项只依赖同一固定父点，所有值可同时选取。规则不产生整数 z，也不自动赋予不同区间间的耦合条件。错误检查数量/顺序/表达式、非法旧坐标、配对乘积或中间差超限均拒绝。

3.12 MenuLift：可选二元菜单

字段为 `menus`、`body`。每 MenuWitness 含 u32 `maximum_index` K、`activation` A、`index` I，以及恰六个 checks，依序证明

$$\mathrm{LE}(-A),\ \mathrm{LE}(A-1),\ \mathrm{INT}(A), \qquad \mathrm{LE}(-I),\ \mathrm{LE}(I-KA),\ \mathrm{INT}(I).$$

A, I 是旧坐标规范仿射式，不要求原本就是单个整数坐标，但必须证明其整数值。

每菜单按 $j = 0, \ldots, K$ 追加 $K + 1$ 个声明整数且界为 $[0, 1]$ 的 z_j，然后依序追加两行、两界均零：

$$\sum_{j=0}^{K} z_j - A = 0, \qquad \sum_{j=0}^{K} j z_j - I = 0.$$

$K = 0$ 仍有两行，第二行是 $-I = 0$，不保存 z_0 的零系数。空菜单列表是恒等扩展，但整数模式与深度要求仍适用。

共同存在。 前提使 $A \in \{0, 1\}$。$A = 0$ 时 $I = 0$，取全零；$A = 1$ 时 $I \in \{0, \ldots, K\}$，只取 $z_I = 1$。所有菜单独立依父点构造，共同可行且各自菜单向量唯一。此局部唯一性不建立完整模型双射；仍须独立完整域单射证明与全输出组合。缺失六前提、不可证明 A/I 整数、尺寸或 KA 中间值超限均拒绝。

3.13 SignedLift：有激活的有界实数正负分解

字段为 `decompositions`、`body`。每 SignedWitness 含精确非负 `capacity` C、旧式 `flow` F、`activation` Z，以及恰五个 checks，依序证明

$$\mathrm{LE}(-Z),\ \mathrm{LE}(Z-1),\ \mathrm{INT}(Z), \quad \mathrm{LE}(F-CZ),\ \mathrm{LE}(-F-CZ).$$

不假设 F 整数。每项按 p, n, u, v 顺序追加四坐标：p, n 连续且非负、无有限上界；u, v 声明整数、界为 $[0, 1]$。每项按序追加四行：

$$p - n - F = 0, \quad u + v - Z = 0, \quad p - Cu \le 0, \quad n - Cv \le 0.$$

前两为两界零的等式；后两仅上界零。$C = 0$ 保留容量行但省去零系数；空列表是恒等扩展，仍检查模式和深度。

共同存在。 $Z = 0$ 时前提强迫 $F = 0$，取全零。$Z = 1, F \ge 0$ 时取 $(F, 0, 1, 0)$，否则取 $(0, -F, 0, 1)$。容量前提保证全部界；当 $F = 0, Z = 1$ 时仍保持 $u + v = 1$，不能删除激活。所有项依同一父点同时选择。负/非规范 C、缺失五前提、错误模式、旧维度攻击及 CZ 中间值超限均拒绝。规则不自动证明实际目标、输出 side constraints 或完整双射。

3.14 SubflowLift：有需求部分的共同有界子流

字段为 `witness`、`body`。SubflowWitness 的 `nodes`、`arcs`、非空 `groups` 与 Flow 的图条件相同。每组含 `source` s、有序 `demands` (t_i, d_i)、每弧 `flows` F_e、`nonnegative` 和 `balances`。需求列表可空，重复终点可用，$d_i \ge 0$；每个终点都须不同于源，包括零需求。flows/nonnegative 必须恰为 m 项，balances 恰为 n 项。令 $B = \sum_i d_i$，$b_s = B$，其他 $b_v = -\sum_{t_i = v} d_i$。依组、弧、节点顺序证明 LE$(-F_e)$ 和 EQ$(\mathrm{div}(F)_v - b_v)$。

每组按弧序追加 m 个连续坐标 g_e，有限界为 $[0, B]$。先按 node 序加 n 个等式 $\mathrm{div}(g)_v = b_v$，再按 arc 序加 m 个 $g_e - F_e \le 0$。前者零常数、两侧为 b_v；后者保留 $-F_e$ 常数且只上界零。G 组共加 Gm 变量、$G(n + m)$ 行。组 a 的变量起点 $N + am$，行起点 $R + a(n + m)$。零弧和空需求合法，仍保留每组所有节点等式。

共同存在。 $B = 0$ 取 $g = 0$。否则加入一个超级终点及每个正需求的终点到超级终点弧，流量为该需求。增广非负流只有源散度 B、超级终点散度 $-B$。从源提取正残余的简单路径到超级终点，减去瓶颈并在旧弧累加到 g。只要源尚有正残余散度，超级终点必可达：否则源的正残余可达集合无正出弧、总散度应不正，却等于正的剩余需求，矛盾。每次永久消去至少一弧，有限步完成；路径权重总和为 B，每简单路径至多使用某弧一次。故同一 g 满足 $0 \le g_e \le F_e$、$g_e \le B$ 和全部需求散度。剩余环流丢弃，自环与断开的圈无需另加条件。各组使用互不相交的新坐标，故共同存在。

它不授权对每弧独立截断：独立 clip 可能破坏流守恒。非法终点、源等于需求终点、负需求、缺失前提、旧维度攻击及预算超限均拒绝。额外 side constraints 与目标关系必须由 body 完整证明，不能从子流引理猜得。

第四章　MCM 与 RPX 二进制格式

当前只接受 MCM 1 与 RPX 11。格式完整定义见 `docs/FORMAT.md`；本章给出与执行规则对应的结构摘要。所有 framing 整数为无符号小端整数。

4.1　公共记录

16 字节头依次为四字节 magic（MCM 或 RPX 加零字节）、u16 版本、u16 零保留位、u64 完整文件长度。不允许尾随字节。tag 为一个 u8 后接三个零字节；vector 为 u32 数量后接各项。名称为 u32 字节数及 UTF-8 字节，模型层再执行 ASCII、长度与唯一性检查。

有理数依次为符号 tag（0 非负、1 负）、u32 分子字节数、u32 分母字节数，然后是无符号小端幅值。分母须正、分子分母互素，无冗余高零字节。零使用正号、空分子幅值和分母一。可选有理数为 u8 0，或 u8 1 后接有理数。仿射表达式为有理常数和 $(\mathrm{u32}, q)$ 项向量；坐标严格递增且系数非零。

MCM payload 依次为模型名、变量向量、行向量、完整仿射目标。变量为名称、u8 整数标志、可选上下界；行为名称、表达式、可选上下侧。

4.2　唯一证书布局

RPX 11 头后直接是 forward Block，再是 reverse Block，没有外围数组计数或可选标志。Block 为 Step 向量后接一个 End。Derivation 为 Step 向量后接 DerivationEnd。类型 tag 为 Le=0、Eq=1、Int=2。Ref 的 tag 依第2.1节顺序为 0–8；除 Exact 无载荷外，其余均后接 u32 索引。可选 Ref 为 u8 0 或 u8 1 后接完整 Ref。

4.3　操作载荷

Step	tag 后的字段顺序
Lin	u32 id，Kind，有理常数，(Ref, q) 向量。
Round	u32 id，Le Ref，Int Ref。
Drop	u32 id 向量。
EqPair	u32 id，positive Ref，negative Ref。
Cases	u32 id，Kind，结论 Expr，split Expr，可选 Int Ref，左右 Derivation。
Plotkin	u32 id，u32 bits，u32 distance，$(\mathrm{u32}, \mathrm{u64})$ 向量，clique Ref 向量。

Step tag 为第三章的 0–5。DerivationEnd 为 0 Witness(Ref) 或 1 Absurd(Ref)。End 0 为 Return（Expr 向量、Ref 向量）；1 为 Split（Expr、可选 Int Ref、左右 Block）；2 为 Absurd（Ref）；3–7 分别为 Flow、Interval、Menu、Signed、Subflow 的 witness 后接 body Block。旧端点 tag 8、9 拒绝。

Flow/Subflow witness 为 u32 节点数、$(\mathrm{u32}, \mathrm{u32})$ 弧向量及组向量。各组为 u32 源、$(\mathrm{u32}, q)$ 需求向量、流 Expr 向量、非负 Ref 向量、平衡 Ref 向量。Interval 为 witness 向量，每项含 lower Expr 向量、upper Expr 向量、checks Ref 向量。Menu 为 witness 向量，每项含 u32 最大索引、activation Expr、index

Expr、checks Ref 向量。Signed 为 witness 向量，每项含 capacity 有理数、flow Expr、activation Expr、checks Ref 向量。具体前提顺序、生成变量和行见第三章。

4.4 结构限制

wire 文件最多 1 GiB；每个 vector 最多 10^7 项，全部计数累计最多 $2 \cdot 10^7$ 项；嵌套深度最多 64；有理幅值每个最多 1700 字节；wire 名称最多 1 MiB。计数还须与剩余字节相容，不能仅凭数量盲目分配。编码器施加同样结构限制。模型、文本导入和路径读取还有各自更窄的限制，见第五章。

第五章　文件边界、API 与资源

5.1　完整文件验收

CLI 用法固定为：

```
rpx verify source.mcm target.mcm proof.rpx
```

不接受额外证明路径、Request 文件或关系选择参数。MPS/LP 须先精确导入 MCM：

```
rpx canonical source.lp source.mcm
rpx canonical target.lp target.mcm
```

导入不使用浮点容差；最大化目标整体取负，保留常数、界、整数声明和全部约束。支持的原始文件方言分别见 `docs/MPS_V1.md` 与 `docs/LP_V1.md`；不支持的构造必须拒绝，不得静默丢弃。

Rust 纯验证函数接收两个 Model、一个 Certificate 及 Limits。文件/字节接口分别为 `api::verify_paths` 与 `api::verify_bytes`；文件 I/O 和计时位于外层，数学执行不读取时钟或环境。结果 status 为 VERIFIED、INVALID、RESOURCE_LIMIT 或 IO_ERROR，CLI 分别退出 0、2、3、4。只有 VERIFIED 表示完整 MI 优化等价。资源拒绝是未完成检查，不表示不等价。

5.2　编写接口

Python 的 Proof 固定拥有 forward 与 reverse 两个 DirectionBuilder。每个方向由 map、步骤和完整收尾组成；作者可用 plan 查看内核生成的义务，用 complete 检查遗漏。计划、扩展布局准备、汇编和序列化都不授予 VERIFIED；最终仍由内核执行两棵证明树。编写器支持稀疏精确表达式、行组合、舍入、分支与五种联合构造，但不搜索 reformulation。

交付文件必须重新读入并与已证明模型精确比较。链式工作流逐阶段验证同一固定目标，并检查相邻模型和文件摘要；链不是内核中的第三类证明对象。

5.3　资源与诊断

路径输入每文件上限为 512 MiB，独立于 codec 的 1 GiB 上限。RPA 12 文本输入上限为 256 MiB；指令、表达式项与列表项合计最多 20,000,000 个节点；嵌套深度最多 128。规范反汇编输出同样受 256 MiB 上限约束。有理数运算中间结果受 65,536 位上限约束；格式幅值另受 1700 字节上限约束。模型名称最多 1024 字节，须非空 ASCII、无控制字符且在各自命名空间唯一。更多精确边界和导入限制见 `docs/API.md` 与格式规范。

work 是确定性收费策略，不等同于 CPU 指令数或物理内存上界。work、live-facts、depth 参数只可控制资源，不可删除数学义务。方向、步骤、义务诊断是尽力报告的位置；统计量与计时不参与数学接受条件。

第六章　文本汇编与编写

RPA 12 是 RPX 11 的行式证明汇编：文本直接降低为同一 Certificate，再编码为 RPX 11 字节，不经过 JSON 中间对象。文本版本与 wire 版本彼此独立；RPA 12 不改变第二章的执行语义或第四章的二进制布局。本章给出语法与命令摘要，完整参考见 `docs/RPA.md`，推理前提见第三章。

6.1　文件、词法与逻辑指令

文件以 `RPA 12` 加换行开头，随后恰为 `.forward` 节与 `.reverse` 节，两节各含一个 Block。Block 由若干 Step 和恰好一个终结指令组成；终结指令之后不得再有 Step，也不得缺少终结指令。

```
RPA 12
.forward
  lin.le f0, r0.hi - 1
  ret (x0) (v0.lo, v0.hi, r0.hi, f0, exact)

.reverse
  ret (x0) (v0.lo, v0.hi, r0.hi, exact)
```

词法规则如下：

- 注释以 ；开始并延续到物理行末；空行忽略。物理行可为 LF 或 CRLF，CRLF 的行尾回车符被剥离；文件末尾换行可选。
- 一条逻辑指令恰好对应一个 Step、End 或 DerivationEnd。逻辑指令可在括号平衡前跨多个物理行，续行不引入新指令；输入缩进不参与语法。
- `left`、`right`、`body`、`end` 是结构行，不对应 wire 节点；每个 Block 恰有一个终结指令。
- 规范反汇编使用两空格缩进、每条逻辑指令一行，并且不输出注释。
- 资源限制：文本最多 256 MiB，指令、表达式项与列表项合计最多 20,000,000 个节点，嵌套深度最多 128。

6.2　引用、坐标与精确表达式

仿射表达式与线性组合的标量都是精确有理数：整数或 `p/q`，分母非零，不得使用十进制或科学计数法。前导 `+/-` 只作用于首项；后续正负号是二元分隔符，因此写 `1 - 2*x0`，不写 `1 + -2*x0`。

引用	含义
`vN.lo`、`vN.hi`、`vN.int`	当前模型的变量下界、上界、整数性事实。
`rN.lo`、`rN.hi`、`rN.eq`	当前模型的行下侧、上侧、等式事实。
`fN`	本 Block 已发布的局部事实。
`aN`	当前分支作用域假设，`N` 为绝对假设深度。
`exact`	精确重言式，不引用任何事实。

坐标写作 `xN`，只允许出现在表达式中；`lin` 只接受引用与有理常数。表达式文法为常数加若干前缀

有理系数的项：

$$\texttt{expr} ::= [\texttt{+}|\texttt{-}]\ \texttt{term}\ \{\,(\texttt{+}|\texttt{-})\ \texttt{term}\,\},\qquad \texttt{term} ::= [\texttt{rational*}]\ \texttt{factor},$$

其中 `factor` 为有理数、坐标或括号表达式。乘法只允许“有理数乘因子”，所以 `x0*x1` 与 `x0/7` 被拒绝；应写 `1/7*x0`。表达式在汇编时经 `Expr::canonicalize` 规范化：常数在前，坐标递增，无零系数。`lin` 的项则保留文本顺序、重复引用与零乘子，常数项求和。空列表写作 `()`；空 `drop` 写作单个词 `drop`。

6.3 Step 指令

助记符	wire	载荷
`lin.le fN, linear`	Lin	输出 Le 事实；前提为 Eq，或乘子非负的 Le。
`lin.eq fN, linear`	Lin	输出 Eq 事实；全部前提须为 Eq。
`lin.int fN, linear`	Lin	输出 Int 事实；前提为 Eq，或整数乘子的 Int。
`round fN, ref, ref`	Round	第二引用为 Le，第三引用为同一线性项的 Int。
`drop [fN, ...]`	Drop	删除显式 ID 列表；列表可为空。
`eqpair fN, ref, ref`	EqPair	两个相反 Le 产生 Eq。
`cases.le fN, expr, split expr [, int ref]`	Cases	两分支 Derivation 共同证明结论。
`cases.eq fN, expr, split expr [, int ref]`	Cases	同上，结论种类为 Eq。
`cases.int fN, expr, split expr [, int ref]`	Cases	同上，结论种类为 Int。
`plotkin fN, bits=u32, dist=u32, vars=(xN:label,...), cliques=(...)`	Plotkin	静态二元码距离上界。

`cases.*` 的指令行后紧接 left derivation、right derivation 与 `end`。Derivation 由若干 Step 和一个收尾组成：`wit ref` 引用结论事实，`absurd ref` 关闭矛盾分支。`wit` 只允许出现在 `cases.*` 分支；Block 中的 `absurd ref` 是 End 而非 DerivationEnd。`plotkin` 的 `label` 为 u64，`cliques` 引用若干 Le 事实，具体条件见第3.7节。

6.4 分支头与终结指令

`split`、`cases.*` 的两支以 `left aN` 与 `right aN` 开头，`N` 是当前绝对假设深度，等于包围该分支的已开启分支作用域数。同一分割的两支共享同一 `N`，嵌套分支递增；汇编器核对头部并拒绝错配。左支假设为分割表达式 $e \le 0$，右支为 $-e \le 0$；带 `int ref` 时右支改为 $1 - e \le 0$，即整数情形 $e \ge 1$。分支假设以 `aN` 引用。

终结指令	wire	结构
`ret (expr, ...) (ref, ...)`	Return	输出向量与位置义务引用。
`split expr [, int ref]`	Split	后接 `left aN` Block、`right aN` Block、`end`。
`absurd ref`	Absurd	关闭当前域。
`interval`	IntervalLift	witness 行、`body`、子 Block、`end`。
`menu`	MenuLift	witness 行、`body`、子 Block、`end`。
`signed`	SignedLift	witness 行、`body`、子 Block、`end`。
`flow nodes=u32`	FlowLift	弧、组、`body`、子 Block、`end`。
`subflow nodes=u32`	SubflowLift	弧、组、`body`、子 Block、`end`。

`ret` 的两个列表按位置解释。检查顺序由 kernel 生成：先按输出变量顺序给出每个有限下界、有限上界、整数性义务；再按输出行顺序给出相等边（两侧有限且相等）或有限下界、有限上界；最后是目标不增不等式。汇编器不读取任何模型，不校验列表长度或表达式是否与义务匹配，也没有命名检查、别名或宏；匹配与数值条件由 verify 阶段的内核判定。因此同一份 RPA 12 可对不同模型汇编，但只有对实际完整模型的验证成功才表示优化等价。

lift 的 witness 行格式固定如下；空 witness 向量允许，`flow`/`subflow` 恰有一个 witness，且所有 `arc` 行先于所有 `group` 行：

```
interval
  lower (expr, ...) upper (expr, ...) checks (ref, ...)
  body
    ...
  end

menu
  K=u32 act=(expr) idx=(expr) checks=(ref, ...)
  body
    ...
  end

signed
  C=signed_q flow=(expr) act=(expr) checks=(ref, ...)
  body
    ...
  end
```

流与子流 payload 各自恰有一个 witness；下面先列弧、再列组：

```
flow nodes=3
  arc 0, 1
  arc 1, 2
  group source=0 commodities=(0:1, 1:2) flows=(
    x0,
    x1,
  ) nonneg=(f0, f1) balance=(f2)
  body
    ...
  end
```

子流把商品列表换成需求列表，其余分组字段相同：

```
subflow nodes=3
  arc 0, 1
  group source=0 demands=(0:1) flows=(x0) nonneg=(f0) balance=(f1)
  body
    ...
  end
```

`menu` 的 `K=` 为最大索引；`signed` 的 `C=` 为有符号容量；`commodities`/`demands` 为 `u32:signed_q` 对列表。

6.5 编写示例

下面的 forward 片段使用穷尽分支；两个右端 `a0` 分别引用各支假设：

```
split x0
left a0
  ret (-x0) (a0, v0.lo, exact)
right a0
  ret (x0) (a0, v0.hi, exact)
end
```

`cases` 分支以 derivation 收尾；该片段在含变量 x_0 的任意模型上可验证，并发布常量事实 $0 \le 0$：

```
cases.le f0, 0, split x0
left a0
  wit exact
right a0
  wit exact
end
```

`interval` lift 先选择区间坐标，再在扩展模型中完成子 Block：

```
interval
  lower (x0 - 1) upper (x0 + 1) checks (exact)
  body
    ret (x0) (v0.lo, v0.hi, exact)
  end
```

实际完整文件见第七章与 `docs/rpx/examples/`。

6.6 规范反汇编与命令行

```
rpx assemble input.rpa output.rpx
rpx disassemble input.rpx output.rpa
```

前两条命令是 RPA 12 文本与 RPX 11 字节之间的双向转换；`compile` 保留为 Python 编译器使用的独立结构化 JSON 桥，它不是 RPA 文本，也不参与行式汇编路径。汇编只做语法与编码，不绑定模型、不接受模型路径参数，也不执行数学检查。反汇编只以一个 RPX 文件和一个输出文本为参数。

规范反汇编是确定性的：首行 `RPA 12`，随后 `.forward`；两空格缩进；每条逻辑指令一行。`ret` 在含缩进不超过 100 列时写成一行，否则两个列表各自多行、每项一行并带尾随逗号；空列表保持 `()`。注释不输出，`Lin` 的项顺序、重复引用与零乘子原样保留。表达式按常数在前、坐标递增输出。因此 `assemble(disassemble(c))` 恢复同一 Certificate。旧版本文本头一律以 `INVALID` 拒绝，不提供就地迁移；带旧版本文本头的 JSON 不再被解析。违反资源限制时返回 `RESOURCE_LIMIT RPA bytes`、`RESOURCE_LIMIT RPA nodes`、`RESOURCE_LIMIT RPA depth` 或 `RESOURCE_LIMIT RPA output bytes`。

第七章　完整可复现实例

三个 CLI 例子位于本手册的 `examples/` 子目录。每个目录包含完整 source.lp、target.lp 与 RPA 12 行式汇编 proof.rpa；汇编得到 RPX 11 字节，反汇编再汇编后与原文件逐字节相同。`examples/python-api/` 另给出使用 Python 编译器 API 构造证明的脚本。

7.1　等式消元

源模型保留等式 $x_0 + x_1 = 1$，目标只使用 $y = x_1$；反向恢复 $x_0 = 1 - y$。

Listing 7.1: 完整源模型

```
Minimize
 obj: x0 + 0 x1
Subject To
 balance: x0 + x1 = 1
Bounds
 0 <= x0 <= 1
 0 <= x1 <= 1
End
```

Listing 7.2: 完整目标模型

```
Minimize
 obj: 1 - y
Subject To
Bounds
 0 <= y <= 1
End
```

Listing 7.3: 两个方向及全部位置义务

```
RPA 12
.forward
  lin.eq f0, -r0.eq
  ret (x1) (
    v1.lo,   ; y lower
    v1.hi,   ; y upper
    f0,      ; objective nonincreasing
  )

.reverse
  ret (1 - x0, x0) (
    v0.hi,   ; x0 lower
    v0.lo,   ; x0 upper
    v0.lo,   ; x1 lower
    v0.hi,   ; x1 upper
```

```
    exact,   ; balance equal
    exact,   ; objective nonincreasing
  )
```

7.2 冗余行与分段恢复

examples/redundant/ 通过已有界证明冗余行，同时两个方向保留完整目标。examples/piecewise/ 使用穷尽分支实现分段恢复；每支仍须独立完成全部输出义务。完整源文件是本手册组成部分，可直接阅读、汇编和验证。

7.3 Python 编译器 API

rpx 的 Rust Core 持有模型空间、作用域、事实、映射和完成状态。作者选择推导；编译器不调用 solver，不搜索证明。ModelBuilder 的 build 封闭建模阶段，finish 封闭证明叶子。不同模型的表达式不能按同名变量自动混用。

```
from rpx import Model, Proof
source = Model.read("source.lp")
target = Model.read("target.lp")
proof = Proof(source, target)
f = proof.forward.map({"y": source.vars["x1"]}, default="same_name")
f.prove("objective.non_increasing", by=-f.row("balance"))
f.finish()
proof.reverse.map({"x0": 1-target.vars["y"],
                   "x1": target.vars["y"]}).finish()
bundle = proof.compile("build/elimination", target_artifact="target.lp")
print(bundle.verify().raise_for_status())
```

map 的默认名称匹配在 Rust 中展开。变量、行、目标、trace 和 certificate 是原生视图，整体物化必须显式请求。Fact 算术产生线性组合；它不与普通表达式混同。连续 split 返回两个 child scope，分别带有 $e \le a$ 和 $e \ge a$ 的假设，边界属于两支。整数 split 要求整性证明，其右支是 $e \ge a + 1$。Cases 的左右推导 scope 分别提交共同结论或矛盾，然后向父级返回一个事实。Interval、Menu、Signed、Flow 和 Subflow 使用类型化输入与生成句柄。父空间可嵌入扩展，生成变量不能逃到父级或兄弟扩展。

封闭证明只 lowering 一次。RPA disassembler 与 RPX encoder 直接消费同一份原生 Certificate。Bundle 保存 source、target、RPA、RPX 和目标原文件；manifest 使用包内相对路径与内容摘要，可搬移。Chain 复制阶段包并校验连接。每次验证使用本次预算，历史 receipt 不替代重新验证。Kernel 结论和 artifact audit 分别显示。

五个可执行脚本与 tutorial.ipynb 位于 examples/python-api/。elimination.py 展示建模和消元，redundant.py 展示冗余行，piecewise.py 展示连续 split，interval_lift.py 展示生成变量，load_models.py 展示文件读取和稀疏映射。脚本默认使用临时交付目录。从仓库根执行：

```
make -C docs/rpx check-compiler-api
```

7.4 统一检查命令

从仓库根执行：

```
python3 tools/check_rpx_manual.py --kernel target/release/rpx
```

当前检查执行 39 个 CLI 命令：验证三个完整例子、汇编以及反汇编/再汇编字节往返，并运行 12 个负例检查，覆盖缺失义务、旧版本头、错误反向映射和资源耗尽。三个例子均各有 9 项总 Return 义务。记录保存在 `examples/verification.json`。汇编阶段拒绝旧版本头/语法错误与执行阶段拒绝数学错误属于不同检查边界。

第八章　性能与证据范围

8.1　RPX 11 简化对比

81 个完整模型对从历史 83 份证书迁移为 81 份双向证书，全部通过新内核。迁移删除附加性质和其专用检查，保留每对模型所需的两个完整 MI 构造。旧文件与收据不覆盖；当前收据另外绑定模型与新证书的字节摘要。

指标	简化前	RPX 11
证书总字节	2,906,200,909	2,057,328,689
总 work	2,281,011,027	1,644,377,506
内核执行总秒	119.02	83.06
进程墙钟总秒	141.10	101.27
最大单进程 RSS（GB）	2.189	2.212

证书字节减少 29.2%，确定性工作量减少 27.9%。执行时间减少 30.2%，墙钟减少 28.2%；最大 RSS 上升约 1.0%，不能宣称内存改善。每个模型对使用两个独立新进程，前后顺序交替，仅一轮且未独占主机。时间与 RSS 是描述性观测；此实验同时改变证书内容与内核检查路径，不能隔离纯解释器开销。从 MCM 开始计时，排除导入和证明生成成本。

原始每案例数据、两个二进制摘要及测量方法见 `audit/validation/mi-opt-simple-20261016/comparison.json`。该数据不与旧版多次测量混合，也不把统计指标当作正确性证据。

8.2　回归证据

当前测试覆盖规则算术、作用域、所有构造族、独立有限 oracle、编解码、资源、模型导入、Python 编写及交付绑定。旧性质和旧 wire 格式专用测试随接口移除，清单与摘要保存在同目录的 `retired-tests.json`；测试数量变化不能直接比较覆盖率。完整 81 案例回放补充小规模单元检查，但不能替代形式化验证或对所有 MILP 的完整性证明。

附录 A　版本边界与迁移

MCM 1 维持原编码。RPX 11 是有意不兼容的简化：证书仅含 forward、reverse，所有根均为完整 MI 域，所有 Return 都检查目标不增。RPA 12 取代 RPA 11 JSON 与 RPA-S 1：文本改为每行一条逻辑指令，直接降低为同一 RPX 11 Certificate，不再有名称检查、模型绑定或模型路径选项。旧 RPA 11 与 RPA-S 1 文本头一律拒绝，不提供就地迁移；RPX 11 的二进制布局与数学语义保持不变。

生产内核没有旧版本解码分支，也没有 Contract、projection、inverse、LP、点、双射、目标层、半空间根限制、ImproveReturn 或 PositiveRefutation 接口。

保留的六种 Step、两种 DerivationEnd 和八种 End 在固定目标下使用。Flow、Interval、Menu、Signed、Subflow 是构造所需输出的方法，不是不同的认证目标。Absurd 和局部分支仍必要，可用于证明不可行域或建立分段映射。

`tools/migrate_mi_opt.py` 通过独立旧二进制读取历史证书，选择两个全域 MI 方向，删除其末尾附加性质义务，输出新证书并要求完整验证。没有合适双向构造的证书不能靠改名迁移。81 个当前案例均已逐项通过，原 83 个历史文件继续保留原字节和原验收含义。

旧版更多性质的数学结果可以作为历史研究记录阅读，不能解释为当前 API 的功能。数值最优解恢复执行器仍未提供；目前验证的是构造的数学合法性。